

birdclef-2026

2位: 疑似ラベリングとタクソン特化モデル による多様性のあるアンサンブル

Rank	2
Team	tennogh
Author	tennogh
Topic ID	704399
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704399>

Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

原文 (Kaggle Discussion) : <https://www.kaggle.com/competitions/birdclef-2026/discussion/704399>

TL;DR:

Public Perch + 改良版 蒸留SED + 自作CNNパイプライン + Insecta (昆虫) 特化モデル

謝辞

このように興味深い年次コンペティションを主催してくれた運営とKaggleに感謝します。実は昨年からぜひ参加したかったのですが、時間の都合がつきませんでした。

私のパイプライン構築に大きく役立った昨年のトップ5解法に感謝します。とくに [2nd place](#) の [XC事前学習済みバックボーン](#) は私の解法の大きな部分を占めています。また [1st place](#) には全体的なロードマップ、insecta特化バックボーンとそのアイデアで助けられました。

今年のコンペからは、public な Perch ノートブックのいくつかのイテレーションを使いました。最終提出でとくに使ったのは [こちら](#) と [こちら](#) です。また public な ["Distilled SED" ノートブック](#) のパイプラインも、最終的にはいくつか変更を加えたうえで使いました。

public なディスカッション、例えば ["What is your best single model LB score ?"](#) や ["How much did pseudo-labeling help you?"](#) にも有益な情報が含まれていました。

総じて、私は比較的初心者なので、これらのリソースは非常に役立ちました。

全体的な戦略

コンペを始めた時点で既に強力な Perch ベースのノートブックが存在しており、自作のCNNモデル（初期の0.8+のものですら）とアンサンブルすると非常にうまく機能することにすぐ気づきました。そこで私の主な目標は、可能であれば自作モデル同士でもアンサンブルの余地を保ちつつ、自分のパイプラインを発展させることになりました。

データセット

私はほぼコンペティションデータのみを使用しました。例外はinsectaモデルで、そこでは [@nikitababich](#) さんの [昨年の追加XCデータ](#) を使いました。

データ分割と検証

自作パイプラインでは、focal クリップ、ラベル付き soundscape、ラベルなし soundscape からなる固定のホールアウトセットを使いました。一部の soundscape のみに出現する種（とくに insecta の全 sonotype）は学習専用（train-only）で、これにより検証はそれらの種に対して盲目（blind）になりましたが、それでもモデルに与えたほうが良いだろうと考えました。

道中で多くの検証戦略を試しましたが、最終的にはどれも信頼できず、主たる指標として LB を使いました。種のサブセットで LB の代理指標（LB proxy metrics）を作ろうとしたり、高信頼度の疑似ラベルデータを使おうとしたりしました。最後まで LB と相関を保っていた唯一の指標は、ラベル付き soundscape 上の AUC スコアでしたが、その相関はごく低いものでした（~0.2）。

メルスペクトログラム

道中でさまざまなメルスペクトログラム（mel spec）を試しました。一般に、hop_length を小さくし n_mels を大きくするとモデルは改善しましたが、その代わり推論時間は増えました（当然の結果です）。また、hop_length を小さくしてからリサイズするほうが、hop_length で直接目標サイズに到達させるよりうまくいくようでした。

ただし、XC事前学習済みバックボーンについては、元の事前学習時の mel spec を使うほうが常に良かったので、ほとんどの提出ではそれらを使いました: 128 mels, 512 hop_length, 2048 FFT, 20 Fmin, 16000 Fmax。

ウィンドウ長

5秒、20秒、そして 20秒 focal / 5秒 soundscape の混合を試しました。結局、@nikitababich さんの2025年解法に似た20秒フレームワークは一貫して成績が振るいませんでした（実装が間違っていたのかもしれませんが）。私はアンサンブルのほとんどのモデルで5秒ウィンドウを使い、例外は20秒/5秒混合のもの1つと、20秒を使う insecta モデル（テストはしていませんが、より原理的に思えました）でした。

バックボーン

最終提出は Efficientnet_v2_s、Efficientnet_b0、NFNet のバックボーン（+ Perch）で構成されています。序盤に多くの異なるバックボーンを試しましたが、NFNet が一貫して最良でした。有望に見えたものもありましたが、疑似ラベリングのフェーズを通して改善させられませんでした（とくに ViT は 0.9 を超えませんでした）。終盤には Efficientnet 系が NFNet を上回り始めました。

疑似ラベリング用に Convnext モデルもいくつか学習しました。Efficientnet/NFNet と比べて最も多様性の高い CNN でしたが、LB は低く推論に時間がかかりすぎたため、アンサンブルには使いませんでした。興味深いことに、PB はこれらのモデルが私が思っていたより強かったことを示しているようです。

いくつかの疑似ラベリングステップ用に PaSST モデルも学習しました。検証指標は良好でしたが、90分の推論時間に収めて LB を確認することができませんでした。

昨年のトップ解法を踏まえ、当初は SED ヘッド + GeM プーリングのシステムを使う予定でした。しかし A/B テストするたびに GeM は実際にはスコアを悪化させたため、平均プーリングをデフォルトにしました。

損失関数

序盤の実験では多くの損失関数を試しましたが、BCE が常に良い結果でした。

しかし後半の疑似ラベリングのラウンドに入ると、当初はモデルの多様性を増やすことを目的として、再び実験を始めました: soft AUC loss は BCE モデルとの相関が低かったものの、LB スコアは低くなりました。

最終的に、私の最良のモデルは AUC loss と少量の BCE loss を組み合わせたものでした (0.25 に落ち着きました)。

データ拡張

波形の mixup、周波数マスキングと時間マスキングは有益で、一貫して使いました。その他のデータ拡張は LB を悪化させました。背景ノイズの拡張は原理的には妥当に思いましたが、うまく機能させることができませんでした。

Perch 蒸留 (distillation)

序盤に Perch 蒸留を試し、バックボーン全体で約 0.02 の改善を得ましたが、同時に自分のモデル間の相関が大きく増加することにも気づきました。そこでアンサンブルの余地を保つため、これを一切使わないことに決めました (それが良い判断だったかどうかは不明です)。

自己ラベリング (self-labeling) のラウンド

昨年のトップ解法の多くと同様に、私はモデルを段階的に改善するために反復的な疑似ラベリングのラウンドを使いました。まだ学習中の身だったので、道中でいくつか新しい設計上の選択をしており、スコアの向上を疑似ラベリングのステップだけに帰することはできません。私が実際に作業した順序どおりに、両方のプロセスを同時に説明しようとするので、このセクションは少し冗長になります。

まず、パイプラインを整えながらコンペティションデータでさまざまなバックボーンを学習することから始めました。2025年5位に触発されて、次に私の最良のモデル (v2_s、NFNet、RegNetY、ViT) と生の Perch を用いて **focal 疑似ラベル** のラウンドを1回行いました。動機は focal クリップ内の二次ラベル (secondary labels) を検出することでした。この段階で、@nikitababich さんの2025年解法の事前学習済み NFnet を使い、hop_length も

512 に下げたところ、LB スコアが 0.917 (PB 0.910) まで上がりました。それ以外では、この段階のスコアは以前 Perch KD で得ていたものとほぼ同じです。

soundscape 疑似ラベルの **Round 1** は、Efficientnet_b4、NFnet、Convnext、PaSST の各モデルに加え、キャリブレーションが大きく異なっていたためゲーティング機構を介した Perch の予測から生成しました。このラウンドでは、20秒の focal クリップに5秒の soundscape チャンクを混ぜて使っていました。疑似ラベル付きチャンクは mixup を通じて追加しました。私の最良のモデルは前ラウンドのバックボーンとコサイン warm restart を使っていました。最良のモデルは NFnet で、LB 0.919 ながら PB 0.926 を記録し、最終アンサンブルに残しました。

疑似ラベルの **Round 2** は、2つの NFnet、Efficientnet_b4、PaSST、Convnext のモデルから生成しました。ここでモデルの改善に苦戦し始めました。[このディスカッション](#)、とくに [@aliozanmemetoglu](#) さんのコメントを読んで、soundscape をラベル付きデータに混ぜるのをやめ、代わりに置換する (50% の確率で、最後のモデルでは 60% に引き上げ) ようにしました。また、後半の実験では5秒ウィンドウのみに落ち着きました。最良のモデルは NFnet で LB 0.922 (PB 0.930) でしたが、Efficientnet と Convnext も同等のスコアを出しました。また、より小さい hop_length とストライドも試し、推論時間を犠牲にして一貫して LB を改善しました。

Round 3 の疑似ラベルは NFnet、Efficientnet、Convnext のモデルで生成し、加えて public な Perch および distilled SED ノートブックのラベルも含めました。ここから昨年2位の XC事前学習済みバックボーンとその mel spec を使い始めました。これらのおかげで私のモデルは LB 0.930 を突破し始めましたが、PB データを見ると、実は初期のモデルのいくつかも同じくらい強かったことがわかります。このラウンドの最良のモデルは LB 0.934 / PB 0.933 を記録した Efficientnet_v2_s ですが、アンサンブルにはそれより少し弱いバージョンを使いました。

Round 4 の疑似ラベルは Efficientnet と NFnet のモデル、および public な Perch と distilled SED のラベルで生成しました。当初はモデルの多様性を加える目的で soft AUC loss を使ったモデルの学習を始めましたが、最終的には LB スコアが最良になる soft AUC + 0.25 BCE の設定に収束しました。これは BCE モデルとの相関がかなり高いものでした。再び20秒モデルの学習も試しましたが、スコアは大幅に低くなりました (これもまた、実装が誤っていたのかもしれませんが)。最良のモデルは LB 0.938 / PB 0.935 の Efficientnet_v2_s で、これが3番目のアンサンブルメンバーです。

Round 5 は、締め切り間際に Round 4 の最良モデルを新しい疑似ラベルでコピー&ペーストしたものです。LB を確認する提出枠がありませんでしたが、そのうちの1つを4番目のアンサンブルメンバーとして追加すると LB と PB がわずかに改善しました。

各ラウンドで使った pseudo-power は、BCE モデルで 1.55、1.0、1.2、1.2、1.2、soft AUC + BCE モデルで 1.0 でした (生成された疑似ラベルの分布から決めました)。

各段階における私の最良スコアのまとめ。すべて単一チェックポイントのスコアで、TTA / 後処理はなしです。一部のチェックポイントは (学習が十分に安定していたときに) SWA 平均を使っています。

Stage	Best LB	PB
Competition data only	0.881	0.869
With Perch KD (later discarded)	0.896	0.891
Focal pseudo-labels round	0.917	0.91
Soundscape pseudo-labels round1	0.919	0.926
Soundscape pseudo-labels round2	0.922	0.929
Soundscape pseudo-labels round3	0.934	0.933
Soundscape pseudo-labels round4	0.938	0.935

Stage	Best LB	PB
Soundscape pseudo-labels round5	0.933	0.932

Insecta（昆虫）モデル

@nikitababich さんのレシピに従い、コンペティションデータと追加XCデータの amphibian（両生類）と insecta（昆虫）のサンプルで学習し（コンペ部分を重み付け強化）、昨年の彼のチェックポイントから開始しました。このモデルは実際には amphibian には役立ちませんでしたが、insecta 用にだけ追加すると LB が 0.002 上がり、5倍の重み付け強化（アンサンブルの50%）で 0.004 上がりました。このモデルをさまざまな設定（自己ラベルまたはアンサンブルラベルを使用）で疑似ラベルで学習することを試しましたが、いずれも LB は悪化しました。

「Public Perch」 アンサンブルメンバー

私は [このノートブック](#) から Perch 埋め込みベースのモデルを採りました。これは再現可能な LB 0.936 / PB 0.930 を記録しました（これ自体が [こちら](#) のコピーでした）。このブランチを改善しようとした試みはすべて失敗したので、結局は集合知（hivemind）の連続した実験を信頼することにしました。多少の LB 過学習は予想していましたが、それでも他のモデルとは多様性があったので、加える価値がありました。

「Distilled SED」 アンサンブルメンバー

@tuckerarrants さんの Distilled SED パイプラインは public ノートブックでも私のモデルでも非常にうまく機能していました（Perch と、私のCNNとで ~0.4 の相関）。そのスコアを改善しようとしたのですが、計算資源と残りの提出枠が乏しかったため、結局やや異なる構成になりました: 蒸留を取り除き、代わりに XC事前学習済み v2_s バックボーンを、softAUCloss+0.25BCE loss（私のパイプラインで最良のスコアを出したレシピ）で使いました。このバージョンは LB 0.929 / PB 0.938 を記録し、それでも残りのモデルとうまくアンサンブルできました。

アンサンブルの意思決定

ホールドアウトしたラベルなし soundscape セットを使ってアンサンブル戦略を練るのに多くの時間を費やしました。最終的には LB スコアが判定基準でしたが、いくつかの指標はアンサンブル性能とそこそこの相関があり、モデル選択の一部に使いました: individual LB（個別LB）、signal-to-noise ratio（SN比）、lift です。Spearman 相関で測定したモデルの多様性は、必ずしも良い予測因子ではありませんでしたが、これは私の評価の実行方法によるアーティファクトかもしれません。insecta 特化モデルについては、すべての指標が最良の重みとチェックポイントの予測に役立ちませんでした（おそらく他のモデルとの負の相関のためです）。

oof データにおけるモデル間の Spearman 相関:

	Perch	SED	r1 NFnet	r3 Effnet	r4 Effnet	r5 NFnet	Insecta specialist
Perch	1.000	0.410	0.428	0.538	0.522	0.494	0.415
SED		1.000	0.437	0.456	0.420	0.390	0.344
r1 NFnet			1.000	0.638	0.529	0.481	0.382
r3 Effnet				1.000	0.792	0.682	0.444
r4 Effnet					1.000	0.706	0.457
r5 NFnet						1.000	0.404
Insecta specialist							1.000

Perch と SED を疑似ラベリングの教師として使うと、それらの相関は ~0.4 から ~0.5 に上がりました。Insecta specialist の相関は insecta のみで計算しています。SED ブランチの数値は、最後のイテレーションで oof を生成しなかったため、わずかに正しくありません。

最終アンサンブル

Model	LB	PB	Note	Solo inference time
Perch	0.936	0.930		23min
Distilled SED-based v2_s	0.929	0.938	AUC+BCE loss	25min
1st round NFNet	0.919	0.926	BCE loss, 20s clips with 5s chunks	17min
3rd round v2_s	0.933	0.928	BCE loss	6min
4th round v2_s	0.938	0.935	AUC+BCE loss	6min
5th round NFNet	0.933	0.932	AUC+BCE loss	9min
Insecta specialist b0	NA	NA	50% of insecta ensemble	8min

一部の I/O と初期化を共有することで、合計推論時間はおよそ 87min になります。

重みはほぼ均等で、r3-r4-r5 ラウンドのモデルは互いに強く相関しているため、それらの重みをわずかに下げました。

個別モデルのスコアは LB と PB の間で大きく動きましたが、それらはほぼ打ち消し合い、全体のスコアは安定していました。最終提出は LB 0.959 / PB 0.960 を記録し、個別モデルのスコアは大きく異なっていたにもかかわらず、私のアンサンブルのほとんどは LB でも PB でも同程度のスコアでした。PB で劣化したアンサンブルは、特定のモデルを過剰に重み付けする実験をしたものでした。したがって私は、均等重みのアンサンブルと、LB 過学習戦略が比較的少なかったことが、最終結果に寄与したと考えています（もちろん、多少の運も加わって）。

後処理

public ノートブックの「sonotype mirroring」と「temporal continuity」のステップを使い、これらは一貫して小さなブーストをもたらしました。その他の手法はすべて失敗するか中立で、とくにあらゆる形式の smoothing はそうでした。それでも public SED パイプラインの gaussian smoothing はそのまま残しました（それをテストする提出枠がなかったためです）。

AIの利用

最近のコンペで、上位スコアの解法の多くが何らかの形で AI を使っているのを見ていましたし、私が参加した頃の Birdclef のディスカッションは [@tom99763](#) さんの実験の話題で持ちきりだったので、恐竜のように古いままではいるのをやめて Claude のサブスクリプションを契約することにしました。

CLI を通じて、コード生成、結果の分析、ディスカッションの要約に使いました。結果的にそれは素晴らしい加速装置となり、これがなければこの順位帯でスコアを出せなかったと思います。ただしその結果として私のコードはかなり肥大化しており、意図しない選択もいくつかあります（例えば私の 20s/5s 混合設定は元々意図したものではありませんでした）。

計算資源の利用

このコンペでは主にリモートの計算資源を使い、Kaggle ノートブックも少し使いました。総コストはおおよそ 200\$ + 2か月分の Claude サブスク（おおよそ 40€）です。

今後追加予定: 推論ノートブック、GitHub リポジトリ、修正…

今後数日にわたって、戻ってきて編集や質問への回答を行う予定です。

Edit 1: [推論ノートブック](#) をモデルの重みとともに追加しました

Edit 2: [GitHub リポジトリ](#)（ハードコードされたパスなど、いくつかの細かい修正を行う予定です）

補足Q&A

質問者（Dhruv Pai Dukle）：上位スコアの提出、素晴らしい仕事ですね。Insecta 特化モデルについて、昨年の優勝者のデータだけを使ったのですか、それともモデルのデータ量を増やすために自分でもデータを収集したのですか？

著者 (tennogh) : そのデータセットとコンペティションデータのみを使用しました。私が始めた頃には XC への過負荷 (overloading XC) についての投稿がいくつかあったので、自分でデータを収集しようとはしませんでした。

質問者 (Dhruv Pai Dukle) : なるほど。私も insecta 用に別のモデルを、自分で抽出したデータ + @nikitababich さんのデータで学習したのですが、LB が悪化しました。おそらく5秒ウィンドウを使ったせいだと思います。40エポックにわたってゼロから学習しました。

著者 (tennogh) : insecta モデルがうまく機能するかどうかを何が決めるのか、私は完全には理解していません。いくつかのチェックポイントは LB に中立か、あるいは有害でした。私が使ったチェックポイントは学習のごく早い段階でピークに達しました。正しいものを選ぶのに役立つ検証シグナルはありませんでした。残念ながら、これを望むほど深く調査する時間はありませんでした。

質問者 (To be happy) : 疑似ラベルを生成する際、使用する教師モデルの出力レンジは似ていますか？私が教師モデルを疑似ラベリングに使ったとき、各モデルの出力レンジが一致しないという問題に遭遇しました。その結果、直接ブレンドすると LB スコアが大きく下がったので、代わりに rank blending を使いました。しかし、結果として得られる出力ラベルは順位スコア (rank scores) になるため、soft label を使うことができません。

質問者 (To be happy) : もう一つ質問です。「2025年5位に触発されて、次に私の最良のモデル (v2_s、NFNet、RegNetY、ViT) と生の Perch を用いて focal 疑似ラベルのラウンドを1回行った」とのことですが、疑似ラベルは学習プロセス中にオンラインで生成したのですか？以前この方法を試したのですが、学習時間が大幅に増えたため諦めました。具体的にどうやったのか教えてください。

「@tuckerarrants さんの Distilled SED パイプラインは public ノートブックでも私のモデルでも非常にうまく機能していました (Perch と、私の CNN とで ~0.4 の相関)」を、ブレンドのためにモデルアンサンブルに加えることを試しましたか？また、これらのモデル間で計算した相関は信頼できる参考になりますか？

著者 (tennogh) :

- キャリブレーションが大きく異なるモデルを同じアンサンブルには使いませんでした。キャリブレーションの異なるモデルを含めるためにゲーティング機構を実装し、いくつかのラウンドで Perch の予測とともに使いました: $\text{ensemble}^* = (0.5 + 0.5 * \text{rank_normalize}(\text{gate_model}))$
- Focal 疑似ラベルはオフラインで生成し、その後それらの上で1ラウンドの学習を行いました (コンペティションのラベルに二次ラベルとして追加)。これは soundscape 疑似ラベリングの各段階と同じロジックです。
- 一部の疑似ラベルアンサンブルでは、オリジナルの Tucker Arrants の予測を使用しました。最終提出アンサンブルでは (write-up で説明したとおり) 変更を加えたバージョンを使用しました。
- 相関は、私のホールドアウト soundscape セット上のクラスごとの Spearman 相関です。これらをアンサンブルの多様性を測るために使いましたが、もっと良いやり方があったかもしれません。

質問者 (To be happy) : 丁寧な返信をありがとうございます。3つ目の質問をもう少し明確にする必要があると思います。私が本来言いたかったのは次の点です。あなたは解法に「@tuckerarrants さんの Distilled SED パイプラインは public ノートブックでも私のモデルでも非常にうまく機能していました (Perch と、私の CNN とで ~0.4 の相関)。そのスコアを改善しようとしたのですが、計算資源と残りの提出枠が乏しかったため、結局やや異なる構成になりました: 蒸留を取り除き、代わりに XC事前学習済み v2_s バックボーンを、softAUCloss+0.25BCE loss (私のパイプラインで最良のスコアを出したレシピ) で使いました」と書いています。

このため私は、「Perch と、私の CNN とで ~0.4 の相関」のせいで蒸留をオフにすることを選んだのだと思いました。そこで質問したいのは、蒸留をオンにしたまま (@tuckerarrants さんの Distilled SED パイプライン)、あなたのモデルと直接ブレンドしたらどうなるか、ということです。その結果はあなたの「oof データにおけるモデル間の Spearman 相関」の結果と一致するのでしょうか。つまり、相関が高いモデルをブレンドするとアンサンブルの効果が低下したり下がったりするのでしょうか。

私の理解が正しいかどうか、確信がありません。

著者 (tennogh) : 主に蒸留をオフにしたのは、そうしたほうがそのパイプラインからスコアを改善しやすかったからです（蒸留と stop_gradient の機構が softAUC や事前学習済みバックボーンと相性が悪かったのです）。

蒸留ありのオリジナルのパイプラインは他のすべてと相関が低かったです。他の相違点（hard label のみ、5-fold 学習）だけで、多様性を持たせるのに十分だったと考えています。

質問者 (M_Saad-sz) : おめでとう。これらのアンサンブルをどうやって時間制限内で走らせたのですか？何か最適化をしましたか？

著者 (tennogh) : ONNX + OpenVino 推論です。同一の mel spec を持つ3つのモデルは、I/O の重複を避けるため同じループ内で実行しています。OOM の問題を避けるため、各モデルの間で重い変数を削除しています。詳細は推論ノートブックを確認してください。

将来的にはモデルごとの推論時間を追加できると思います。例えば XC事前学習済み Nfnet でおよそ10分、v2_s でおよそ7分です。